

מסמך אפיון טכני – מיפל (Maple Garden) (Events)

פרויקט: מערכת ניהול אולם אירועים כשר

גרסת מסמך: 1.0

תאריך: 8 ביולי 2026

מבוסס על: קוד המקור בפרויקט `Maple-Garden-Events`

תקציר מנהלים

מיפל היא מערכת **Full-Stack** לניהול אולם אירועים: לוח שנה דו-תאריכי, אופציות והזמנות, הפקה, קבלה LIVE, תשלומים, חוזים, משוב לקוחות ותקשורת אוטומטית. הארכיטקטורה בנויה כ-**Monorepo** עם שני רכיבים עיקריים – אפליקציית אדמין ב-React ושרת API ב-Node.js – המתקשרים ב-**REST** וב-**WebSockets** מול בסיס נתונים **PostgreSQL**.

המערכת מיועדת לצוות פנימי קטן (מנהלים, מזכירות, הפקה, קבלה) ולדף משוב ציבורי חד-פעמי ללקוחות. היעד התפעולי הוא **אמינות, מניעת כפילויות, וסנכרון בזמן אמת** – לא עומס של אלפי משתמשים ציבוריים.

1. חומרי גלם טכנולוגיים (Technology Stack)

1.1 מבנה הפרויקט

```
Maple-Garden-Events/  
├─ client/           # Frontend - React SPA  
├─ server/           # Backend - Express API  
├─ tests/e2e/        # בדיקות Playwright  
├─ PRD.md            # מסמך אפיון עסקי (נפרד)  
└─ package.json      # סקריפטים ראשיים (dev, e2e)
```

1.2 שכבת (Client) Frontend

רכיב	טכנולוגיה	גרסה	תפקיד
Framework	React	x.19	ממשק אדמין RTL בעברית
שפה	TypeScript	x.6	Type-safety מלא
Build Tool	Vite	x.8	פיתוח מהיר + Bundle לפרודקשן
Routing	React Router	x.7	ניווט בין מודולים
State / Cache	TanStack Query	x.5	שליפת נתונים, cache, invalidation
State מקומי	Zustand	x.5	מצב UI נקודתי
HTTP Client	Fetch + Axios	—	קריאות API מאובטחות
Real-time	Socket.io Client	4.8	סנכרון לוח שנה, הזמנות, LIVE
Auth	react-oauth/google@	0.13	התחברות Google בצד לקוח
Charts	Recharts	x.3	סטטיסטיקות משוב
OCR (צ'קים)	Tesseract.js	x.7	זיהוי טקסט מצילום צ'ק
חתימה דיגיטלית	react-signature-canvas	—	חתימת לקוח בחוזה/LIVE
ניטור	sentry/react@	x.10	דיווח שגיאות בצד לקוח
UI	CSS Modules + רכיבי UI פנימיים	—	Design System עצמאי

מודולים כבדים (הזמנות, הפקה, הגדרות) נטענים ב- `React.lazy` לשיפור זמן טעינה ראשוני.

1.3 שכבת Backend (Server)

רכיב	טכנולוגיה	גרסה	תפקיד
Runtime	Node.js	—	שרת אפליקציה
Framework	Express	x.5	REST API
שפה	TypeScript	x.6	לוגיקה עסקית מוקלדת
ORM	Prisma	x.6	גישה ל-PostgreSQL + מיגרציות
DB	PostgreSQL	—	אחסון נתונים (יעד: AWS RDS)
Auth	Google OAuth + JWT	—	אימות משתמשים מורשים
Validation	Zod	x.4	ולידציית קלט ב-API
Real-time	Socket.io	4.8	Broadcast לכל הלקוחות המחוברים
PDF	Puppeteer	x.25	הפקת חוזים וטפסי הפקה
Email	Nodemailer	x.8	תזכורות, משוב, התראות
WhatsApp	Green API (REST)	—	הודעות ללקוחות ומנהל
חשבוניות	EasyCount API	—	הפקת קבלות/חשבוניות
לוח עברי	hebcap / @hebcap/core	—	תאריכים עבריים, חגים, צומות
Scheduling	node-cron	x.4	תזכורות, גיבוי, משוב, ברכות
Logging	Winston + Daily Rotate	x.3	לוגים מובנים + רוטציה יומית
Security	Helmet, CSRF, Rate Limit	—	הגנות HTTP
ניטור	Sentry (Node)	x.10	שגיאות + traces
Storage	AWS SDK S3	x.3	גיבויי DB (מיושם); קבצים – בתכנון

רכיב	טכנולוגיה	גרסה	תפקיד
Uploads	Multer (memory)	x.2	קבצים מצורפים (ברכות וכו')

1.4 תשתית ופריסה (יעד)

רכיב	שירות מיועד
אפליקציה	AWS ECS / App Runner
בסיס נתונים	AWS RDS PostgreSQL (Managed)
קבצים סטטיים (Frontend)	S3 + CDN
גיבויים	S3 (SSE) – מיושם ב- <code>databaseBackup.ts</code>
ניטור	Sentry
בדיקות	Playwright E2E

1.5 מודל נתונים מרכזי (PostgreSQL / Prisma)

תפקיד	ישות
אופציה / הזמנה – ישות מרכזית	Booking
תאריך בלוח שנה + סטטוס	EventDate
טופס הפקה + תפריט + שולחנות	EventForm
קבלה LIVE	EventCheckIn
משוב לקוח (Token ייחודי)	Feedback
הגדרות תמחור (Singleton)	SystemSettings
רשימת אימיילים מורשים	AuthorizedUser
תפריט	Dish / MenuCategory
ברכות מתוזמנות	ScheduledGreeting

אילוצי ייחודיות קריטיים

- double-booking – @@unique([calendarDateId, timeSlot]) : Booking
- @unique : Booking.eventCode
- @@unique([bookingId, clientSide]) : Feedback
- @unique : EventDate.date

2. קשרי גומלין ואינטגרציה (Architecture & Data Flow)

2.1 תרשים ארכיטקטורה כללי



2.2 זרימת תקשורת Client ↔ Server

REST API

- כל הקריאות עוברות דרך `secureFetch` עם `credentials: 'include'`
- בסיס הנתונים: `/api` (או `http://localhost:5000/api` בפיתוח)
- פורמט: JSON
- בפרודקשן: השרת יכול לשרת גם את `client/dist` (SPA) כאשר `SERVE_CLIENT=true`

קבוצה	נתיב	תיאור
Auth	<code>/api/auth</code>	login, refresh, logout, me, authorized-users
Calendar	<code>/api/calendar</code>	תאריכים, חסימות, לוח עברי
Bookings	<code>/api/bookings</code>	CRUD, אופציות, תוספות PDF, LIVE חוזה
Options	<code>/api/options</code>	ניהול אופציות
Event Forms	<code>/api/event-forms</code>	הפקה, שולחנות
Settings	<code>/api/settings</code>	תמחור, שירותים, צוות
Feedback	<code>/api/feedback</code>	ניהול + שליחה ציבורית
Check-In	<code>/api/check-in</code>	קבלה LIVE
Menu	<code>/api/menu</code>	קטגוריות ומנות
Kashrut	<code>/api/kashrut</code>	תעודות כשרות

WebSockets (Socket.io)

- חיבור מאומת באמצעות JWT מ-cookie (`socketAuthMiddleware`)
- שינוי בשרת מפעיל `queryClient.invalidateQueries` בכל הלקוחות – ללא רענון דף

אירוע	מקור	פעולה ב-Client
date-updated	שינוי בלוח שנה	Invalidate calendar, bookings
booking-updated	שינוי הזמנה	Invalidate bookings, calendar
settings-updated	הגדרות	Invalidate settings, kashrut
event-forms-updated	הפקה	Invalidate event-forms
check-in-updated	LIVE	Invalidate check-in
feedback-updated	משוב	Invalidate feedback-admin, feedback-stats
menuUpdated	תפריט	Invalidate menu

2.3 אבטחת מידע

מנגנון	יישום
אימות	JWT → AuthorizedUser מול בדיקה מול Google OAuth
Session	HttpOnly cookies ב- (7 ימים) Refresh Token + (1 שעה) Access Token
CSRF	Header + maple_csrf Cookie X-CSRF-Token לכל פעולות כתיבה
Authorization	requireAuth middleware על רוב ה-routes
Rate Limiting	500 בקשות/15 דק' (API), 150 כתיבות, 20 refresh, 60 login
HTTP Headers	Helmet
CORS	רשימה לבנה: localhost:5173 + CLIENT_URL
ולידציה	Zod schemas לכל endpoint רגיש
Socket Auth	JWT חובה לפני חיבור WebSocket

2.4 אינטגרציות חיצוניות

Google OAuth

1. המשתמש מתחבר ב-Google בצד הלקוח
2. ID Token נשלח ל- `POST /api/auth/login`
3. השרת מאמת מול Google, בודק `AuthorizedUser`, מנפיק cookies

EasyCount (חשבוניות)

- מצבי הפעלה: `live` | `sandbox` | `simulation` | `off`
- ברירת מחדל: `simulation` (ללא credentials)
- סטטוס נשמר ב- `Booking.easycountStatus`
- Fallback: המשך תפעול גם בכשל API

WhatsApp (Green API)

- שליחת תזכורות (צ'ק ביטחון, טופס הפקה, תפריט)
- בדיקת `existsWhatsapp` לפני שליחה
- Fallback לאימייל במקרה כשל

אימייל (Nodemailer)

- תזכורות תפעוליות וכספיות
- משוב אוטומטי לאחר אירוע
- התראות מנהל (כשרות, חובות)

Puppeteer (PDF)

הפקת חוזים חתומים וטפסי הפקה להורדה.

Tesseract (OCR)

זיהוי פרטי צ'ק ביטחון מצילום (צד לקוח).

- **מיושם:** גיבוי DB דחוס (`S3` → `.tar.gz` → `pg_dump`)
- **בתכנון:** העלאת חתימות, צ'קים, גלריה – Presigned URLs + SSE

2.5 תהליכי רקע (Cron Jobs)

תזמון	משימה
09:00 יומי	תזכורות צ'ק ביטחון, טופס הפקה, חובות
08:00 יומי	התראת תוקף תעודות כשרות
כל 10 דקות	יצירת משוב + שליחת קישור לאחר אירוע
כל דקה	ברכות מתוזמנות
03:00 יומי (אופציונלי)	גיבוי DB ל-S3 (<code>BACKUP_ENABLED=true</code>)

2.6 זרימת נתונים לדוגמה – שמירת הזמנה

1. `Client` משתמש בוחר תאריך בלוח שנה
2. `POST /api/bookings` עם פרטי לקוח ומשבצת
3. `Server`:
 - a. `Zod` ולידציה
 - b. `bookingDateValidation` בדיקת זמינות משבצת
 - c. `Booking + EventDate` יצירת `Transaction`
 - d. `race condition` מונע `DB @unique` אילוצ
4. `emitBookingUpdated()` → `Socket.io`
5. לוח שנה מתעדכן → `invalidateQueries`: כל הקוחות

2.7 מצב יישום מול תכנון

יכולת	סטטוס בקוד
4 – RBAC תפקידים	מתוכנן – כרגע כל משתמש מורשה מקבל <code>role: 'manager'</code> ב-JWT
Fallback Offline (LIVE)	מתוכנן – טרם מיושם בקוד הלקוח
S3 Presigned URLs לקבצים	מתוכנן – כרגע העלאות מקומיות / memory
Twilio WhatsApp	מתוכנן – מיושם Green API

3. מטרות על ויכולות ביצועים (Scale & Performance)

3.1 יעד סופי של המערכת

ממד	יעד
משתמשים פנימיים	5–15 משתמשים במקביל (צוות אולם)
לקוחות קצה	דף משוב ציבורי – עומס נמוך, חד-פעמי
אירועים פעילים	מאות הזמנות בשנה (2026–2027+)
זמינות	99.5%+ (שעות עסקיות + ימי אירוע)
אמינות קריטית	0 0 double-booking, אובדן חתימות/תשלומים
זמן תגובה	> 2 שניות לרוב פעולות UI

הערת הקשר: דרישה של 1,000 משתמשים בו-זמנית רלוונטית בעיקר לפלטפורמות ציבוריות. מיפל היא מערכת B2B פנימית – פרופיל העומס האמיתי הוא עשרות בקשות בדקה בשיא (יום אירוע LIVE), לא אלפי חיבורים. הארכיטקטורה הנוכחית מתאימה לפרופיל זה.

3.2 יכולות ביצועים קיימות בקוד

בסיס נתונים

מנגנון	תיאור
אילוץ ייחודיות	<code>calendarDateId + timeSlot</code> – מניעת כפילות ברמת DB
בדיקת שרת	<code>validateSlotAvailability</code> לפני INSERT
טיפול ב-race	<code>bookingSlotGuard</code> – זיהוי P2002 והחזרת 409
Retry	<code>withDbRetry</code> – עד 5 ניסיונות על שגיאות חיבור
Pagination	רשימות מוגבלות ל-50 (מקסימום 100)
Transaction options	<code>maxWait: 15s, timeout: 30s</code>

שכבת אפליקציה

מנגנון	תיאור
TanStack Query	<code>staleTime: 60s, refetchOnWindowFocus: false</code>
Lazy Loading	Code-splitting למודולים כבדים
Rate Limiting	הגנה מפני abuse ו-DDoS בסיסי
Socket.io Broadcast	עדכון מיידי ללא polling
Request Logger	מידת <code>durationMs</code> לכל בקשה

לוגינג וניטור

כלי	יישום
Winston	לוגים JSON, רוטציה יומית (14 יום info, 30 יום error)
Request Logger	method, url, status, duration, IP
Sentry	שגיאות +500, exceptions, traces (10% sample)
Error Handler	לכידה מרכזית + דיווח Sentry

יציבות וגיבוי

מנגנון	תיאור
גיבוי DB	pg_dump → דחיסה → 30 retention S3, יום
EasyCount Fallback	מצב simulation + סטטוס FAILED
WhatsApp Fallback	retry + אימייל חלופי
E2E Tests	Playwright – שלד Golden Path (בהשלמה)

3.3 הערכת עומס לפי תרחישים

תרחיש	עומס משוער	התאמה
יום עבודה רגיל (3-5 משתמשים)	1-5 req/sec	✓ מעל ומעבר
סגירת הזמנה (פיק)	10-20 req/min	✓
יום אירוע LIVE (2-4 מכשירים)	עדכוני check-in + sockets	✓
שליחת ברכה המונית	100-500 אימיילים	⚠ תלוי SMTP rate limits
1,000 משתמשים בו-זמנית	~100-500 req/sec	✗ דורש הרחבה

3.4 מסלול הרחבה לעומסים גבוהים

שכבה	פעולה מומלצת
Compute	Horizontal scaling – מספר instances ב-ECS + Load Balancer
WebSockets	Redis Adapter ל-Socket.io (סנכרון בין instances)
Cache	Redis ל-session, הגדרות, לוח שנה חודשי
DB	Read Replicas, אינדקסים ממוקדים
CDN	CloudFront ל-assets סטטיים
Queue	SQS / BullMQ לברכות, משוב, EasyCount (async)
Load Testing	k6 / Artillery – לפני go-live מורחב

3.5 KPIs טכניים מומלצים לניטור

מדד	סף התראה
Response Time P95	< 3 שניות
Error Rate (5xx)	< 1%
DB Connection Pool	< 80% utilization
Socket Connections	< 50 בו-זמנית (חריג לאולם)
Cron Job Failures	כל כשל → Sentry alert
Backup Success	כשל יומי → התראה

4. סיכום והמלצות לצוות

נקודות חוזק

1. הפרדה ברורה בין Client / Server / DB עם API מובנה
2. מניעת double-booking בשלוש שכבות: ולידציה, אילוץ DB, טיפול ב-conflict
3. סנכרון בזמן אמת לצוות מרובה משתמשים
4. אבטחה מרובת שכבות: OAuth, JWT, CSRF, Rate Limit, Helmet
5. תפעול אוטומטי: Cron jobs, גיבויים, התראות
6. ניטור: Winston + Sentry + request duration logging

פערים להשלמה לפני Go-Live

1. RBAC מלא – הוספת `role` ל- `AuthorizedUser` + `requireRole` middleware
2. `Fallback Offline` במסך `localStorage + sync` – LIVE
3. `S3 לקבצים` – חתימות, צ'קים, גלריה (Presigned URLs)
4. השלמת בדיקות `E2E` – Playwright Golden Path עם `auth mock`
5. `Load test` – אימות עומס יום אירוע (10–20 משתמשים מקבילים)

מסמך זה הופק על בסיס ניתוח קוד המקור בפרויקט *Maple-Garden-Events*. מסמך עסקי נפרד:
.PRD.md / PRD.html

מיפל – גן אירועים בעיר · מסמך אפיון טכני 8 · v1.0 ביולי 2026